

Teaching extreme programming to large groups of students

Görel Hedin *, Lars Bendix, Boris Magnusson

Department of Computer Science, Lund Institute of Technology, Ole Römers väg 3, Lund, Sweden

Received 1 September 2002; received in revised form 1 February 2003; accepted 1 September 2003

Available online 30 December 2003

Abstract

We find the extreme programming methodology highly suitable for introducing undergraduate students to software engineering. To be able to apply this methodology at a reasonable teaching cost for large student groups, we have developed two courses that work in tandem: a team programming course taken by more than 100 students, and a coaching course taken by around 25 students. In this paper we describe our view of how extreme programming fits into the software engineering curriculum, our approach to teaching it, and our experiences, based on two years of running these courses. Particularly important aspects of our set up include team coaching (by older students), fixed working hours, and colocation during development. Our experiences so far are very positive, and we see that students get a good basic understanding of the important concepts in software engineering, rooted in their own practical experience.

© 2003 Elsevier Inc. All rights reserved.

1. Introduction

Teaching software engineering is difficult since many of the practices are motivated by large projects and large organisations of which the students have little or no experience. Like many other universities, we have previously been running an introductory software engineering course based on the waterfall model, using a small project to illustrate basic concepts such as requirements analysis, specification, design, implementation, testing, documentation, and delivery. This traditional set up, with little or no support from faculty during the project, has always been problematic. In particular, it has been difficult to motivate and enforce all the practices for such a small project. With only one iteration, the waterfall model has no place for students to improve. The result has too often been that a few students on each team have been doing most of the work, drowning in details and deadlines rather than grasping concepts, and the rest of the team has learned even less.

The emergence of the eXtreme Programming (XP) methodology (Beck, 1999, Beck, 2000), immediately appeared to us as a much more viable way of both practicing and teaching software development. XP is a so-called agile methodology (Manifesto, 2001) promoting a highly iterative work model with the aim of producing high-quality software and allowing quick adaptation to changing requirements. It is centered around the 12 practices of: Planning Game, On-site Customer, Pair Programming, Collective Code Ownership, Continuous Integration, Small Releases, Metaphor, Simple Design, Coding Standard, Testing, Refactoring, and 40-h Week. In a typical XP project the team may consist of around 6–12 developers, a coach, and a customer that is “on-site” and therefore available for informal discussions. The development work will typically be organised in short iterations of around 2–3 weeks between which planning meetings are held. The intervals between releases is also short, for some projects on a daily basis and for others after a few iterations, the goal being to let the customer give feedback as often as possible and to be able to steer the project direction.

In the spring of 2002, we introduced XP into the curriculum of our 4.5 year computer science and engineering program, replacing the earlier waterfall-based course. Among the questions we then faced were: How

* Corresponding author.

E-mail addresses: gorel@cs.lth.se (G. Hedin), bendix@cs.lth.se (L. Bendix), boris@cs.lth.se (B. Magnusson).

does the XP methodology fit into the curriculum? How can we adapt XP to a teaching setting? How can we handle this methodology in our fairly large classes of over 100 students?

Central to our approach has been to develop two courses in tandem: a Team Programming course based on XP, and a Coaching course, where older students serve as coaches for teams of students taking the Team Programming course.

Although our gut feeling is that all of the XP practices are very valuable, our main goal has not been to teach XP per se, but to use XP as a vehicle for teaching the basic concepts in software engineering. In running a practical project there is the need for following a method, and in using XP as the basis we have found it easy to give the students a solid basic understanding of not only the important concepts mentioned above, but also of, e.g., iterative development, configuration management, and team communication. In particular, the highly iterative nature of XP, where the participants get rapid feedback at all levels, makes it ideal in a teaching situation. As a side effect, the Team Programming course also gives the students practical skills in testing, configuration management, and refactoring, skills that they can immediately use in their projects in later courses. We find such skills to be much easier to teach in the new XP-based set up of the course.

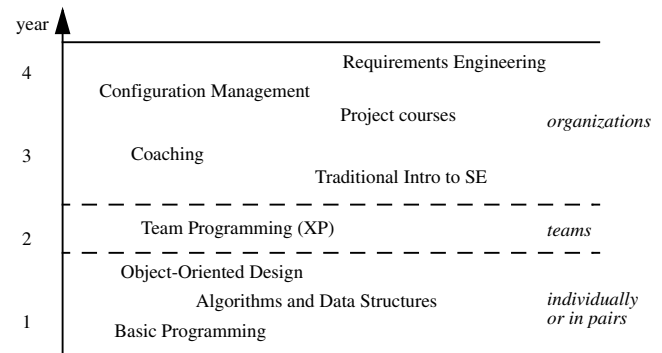
In two earlier papers, (Hedin et al., 2003a,b), we reported on our initial experiences from the two courses. This paper is based on these papers and presents the combined approach, as well as our experiences from running both courses twice. The remainder of this paper is organized as follows: In Section 2 we discuss the placement of XP in the curriculum. Section 3 describes our main set up of the two courses and how they work together. Sections 4 and 5 describe detailed practices and experiences from the respective courses. Section 6 discusses related work, and Section 7 provides our conclusions.

2. Placing XP in the curriculum of software engineering

There are many choices for when to introduce XP in teaching. XP can provide a lot of concrete valuable practices and experience that is valuable in other courses, and we therefore find it important to place it early in the curriculum.

The figure below shows how our Team Programming and Coaching courses fit into the larger picture of the curriculum where many different aspects of software engineering and technology are treated. We view the curriculum as being composed of three layers. At the bottom layer there are first and second year courses teaching basic object-oriented programming, algorithms and data structures, and object-oriented design. In these

courses the students work individually or in pairs, producing small programs of at most a couple of hundred lines of code. At the middle layer we find the Team Programming course which is focused on programming in small teams (8–10 developers). At the top layer we find courses that address software development issues in larger organisations and/or more advanced topics.



The introductory software engineering course that we used to teach (based on the waterfall method) is really about software development in organizations, and since the students only had experience from individual work, it thus created a mismatch that was hard to bridge. In our new curriculum, the XP course (Team Programming) serves as a practical introduction to software engineering, giving the students experience from a small, but complete project that gives a concrete illustration of all the basic concepts usually taught in software engineering courses. When the students later take courses on more traditional software engineering topics, they have a personal practical experience that they can relate to and build from, allowing them to easier appreciate, understand, and critically discuss the concepts taught at the third level.

For example, after the Team Programming course, the students take a course giving an overall introduction to software engineering, based on a standard textbook covering the traditional methods and phases. Here, the students can relate to the particular method they know in detail (XP). In the Configuration Management course, different collaboration strategies are taught, like split-combine, turn-taking, and copy-merge, different branching strategies, etc. Here, the students can relate to their experience from the simple unbranched copy-merge strategy they have used in the XP project.

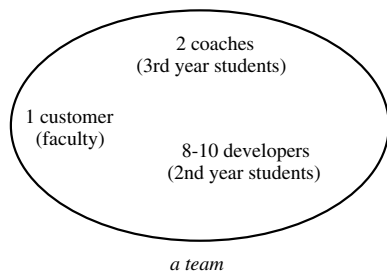
In the top layer we also find the Coaching course, where the students learn about team coaching and software architecture. Each student in the Coaching course acts as a coach for one of the teams in the Team Programming course. At this level we also find various courses which include implementation projects, e.g., compiler construction, real-time programming, etc., where the students can make use of the skills they have learnt in the Team Programming course.

Could the XP methodology be introduced even earlier than we do? Certainly, many elements of XP can be trickled down into the introductory courses at the individual/pair level, for example, Pair Programming, Testing, Refactoring, and Simple Design. Others are more related to team programming and are better introduced at that level, e.g., Planning Game, Small Releases and Continuous Integration. However, even if some of the practices are beneficial to introduce earlier, they will all bring out more issues when applied in a team. For example, switching partners within a team brings out more from Pair Programming than the experience that can be gained from Pair Programming with a fixed partner on a small introductory problem.

3. Set up of the courses

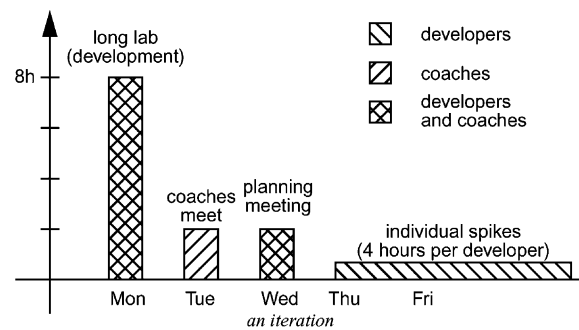
The two XP courses are given in parallel during two study periods (our academic year is divided into 4 study periods of 7 weeks each), and the students take other courses in parallel as well. The Team Programming course is mandatory for 2nd year students and is taken by slightly over 100 students each year. The course is a pass/fail course giving 4 credits (students take 40 credits a year). The Coaching course is elective for 3rd and 4th year students and is taken by around 25 students each year. This course is also a pass/fail course giving 6 credits. The students are studying for a Master of Science in Computer Science and Engineering, and they are used to quite dense and intensive courses.

In both courses the first period is a theory part and the second a project part. The coaching students also do an in-depth study during the second period. During the project part the students split into 12 parallel teams, each consisting of 8–10 developers (students taking the Team Programming course) and 2 coaches (students taking the Coaching course). Each team also has a person from faculty acting as Customer.



The project is run as a series of 6 weekly iterations. Iterations start on Wednesdays with a 2-h planning meeting where the team follows up on the previous iteration and plans for the next one. The following Monday, the whole team (developers and coaches) meet in one of the labs for 8 h, a “long lab session”, of program development. In the meantime, each developer is

expected to spend around 4 h of “spike time” on experimenting and studying particular issues. Spikes can be done individually, in pairs, or in larger groups. However, actual program development (which is checked into the common team repository) takes place only during the long lab sessions. All 24 coaches meet each Tuesday to discuss experiences from the previous iteration and to get advice from faculty for the upcoming iteration.



All teams develop essentially the same product (a system for tracking races in the motorcycle sport Enduro) and are given the same set of user stories (customer requirements). However, they are not expected to complete all the stories. The focus is rather on completing a reasonable amount, while maintaining code quality by adhering to the XP process.

In most other course projects we give fixed requirements that the implemented system should fulfil. The Team Programming course is very different. Here we give the students a fixed time box (planning time, spike time, and long lab time), and expect them to do the best of this time. Completing stories is important, but not at the expense of code quality, tests, team communication, and peer learning. We find this approach absolutely essential in order to make sure that the students practice various skills rather than focusing on completing functional requirements in the shortest possible time.

How can we grade a course that is time boxed and builds so much on teamwork? We grade it only pass/fail, and for passing they are required to be actively participating in all the scheduled activities. Students can apply for an exemption from the scheduled activities due to illness or exceptional circumstances. In order to make up for such absence, the students get additional spike time tasks that are valuable for their team and which are presented to both the team and us.

3.1. Details of the team programming course

The theory part of the Team Programming course contains six lectures of 2 h each covering: introduction; overview of XP practices; testing and pair programming; configuration management; simple design, refactoring, and architecture; and planning and estimation. At the

seventh lecture the students are given a 1-h quiz that they must pass in order to be allowed into the project part of the course. Also, the project and product to develop are introduced at this lecture.

During the theory part there are also four lab sessions of 2 h each, introducing the main practices and the tools. These labs cover: planning (which is conducted as a so called Extreme Hour (Auer and Miller, 2002)); test-driven development and pair programming (using Junit (Beck and Gamma, 2002)); configuration management (using CVS); and refactoring (using the Eclipse Java Development Tools). During these labs the students work in lab groups of 15–20 students, presence is compulsory and a short quiz is done at the beginning of each lab in order to ensure that the students are sufficiently prepared to get full benefit from the lab. The Extreme Hour is done in a classroom, whereas the other three labs are done in the computer lab working in pairs.

In the project part, presence is compulsory in the planning meetings and long labs. In the last (7th) week of the study period, the resulting products are demonstrated and evaluated. This is done by peer evaluation, letting each team try out another team's product and review their code, technical documentation and user manual. These reviews are presented orally in groups of three teams at a time.

As main literature we chose the book "XP Installed" (Jeffries et al., 2001), and supplemented this with some articles (Beck, 1999; Beck and Gamma, 2002; Wake, 2000; Babich, 1986). We find the book a good practical introduction to XP, but it is very different from the normal textbooks that students usually have. It is aimed at practitioners and explains a particular method (XP) without any broader outlook or objectivity. We solve this by presenting the book as a "handbook" for the particular method used, rather than the objective final truth.

3.2. *The Coaching course*

The students in the Coaching course have the Team Programming course as a prerequisite and thus have a good understanding of XP already. The teaching goals of the Coaching course are to get a deeper understanding of the XP process, to develop practical skills for coaching and leading a development team, and to practice ideas from software architecture on a small scale.

3.2.1. *Theory part*

The topics covered in the theory part are focused on design and architecture, covering patterns for test-driven development (Beck, 2002), bad smells and refactorings (Fowler, 1999), design patterns (Gamma et al., 1993), First Iteration and Metaphors (Wake, 2001) and multiple architectural views (Kruchten, 1995). We also in-

clude some material on coaching as such, including articles from (Demarco and Lister, 1999, Whitmore, 2002). However, in our view, coaching is mainly taught through practice, reflection, and sharing of experience, and the readings within this area serve mainly to give some initial ideas. The theory part is organized as a number of 2-h discussion seminars. For each seminar there is typically a couple of articles to read and a short homework of preparing some discussion input. During the seminar, the participants typically divide into small groups discussing focused topics, and present their results to the other groups.

3.2.2. *First iteration*

The practical part of the Coaching course starts by a one-day lab session when each pair of coaches develop the First Iteration of the product for their team, i.e., an executable skeleton of the core of the product is developed (Wake, 2001). This constitutes an initial architecture for the product, and is used as the starting point for the developers on the Team Programming course. This fits with the project scenario of the 8–10 developers as "newly hired staff" that ramp up the initial team consisting of a pair of senior developers (the coaches).

3.2.3. *Coaching*

During the 6 iteration XP project, the students on the Coaching course serve as coaches and project team leaders. The coaches lead the planning meetings and they coach actively during the one-day long lab iterations. They also have the responsibility of tracking the evolving architecture of the product and to make sure it is discussed within the team so that all developers share the big picture. The coaches meet for 2 h once a week together with a faculty supervisor in order to share experiences from their coaching, to get advice, and to reflect on their own role.

3.2.4. *In-depth study*

During the project, the coaches also perform an in-depth study of a specific topic of their own choice. Each study is documented in a small report written in article style. Examples of selected topics include both technical topics, like specific testing techniques and tools, and softer topics, like pair programming experience or analysis of team roles. Many of the coaches take advantage of their team in the study for conducting small experiments and surveys.

3.3. *Bootstrapping the courses*

The first time we gave the Team Programming course we naturally had no students with XP experience to enrol on the Coaching course. In order to bootstrap the courses we had to train a first batch of coaches. To this end, the people following the first Coaching course in-

stance was a combination of faculty, graduate students, and a few handpicked senior undergraduate students. This allowed us to train staff at the department as well as try out the Coaching course on a few experienced students. The student coaches more than fulfilled our expectations, not the least considering team building. In this first instance of the Coaching course, we used the same format as described above, with a theory part, a coaching part, and an in-depth study. The contents of the theory part were, however, different, focusing on introducing the basic XP practices, rather than focusing on architecture.

3.4. Comparisons between the teaching project and real world XP projects

Naturally, there are many differences between our teaching projects and real world XP projects. The teaching project is very scaled down in size, each development iteration being a single day of development as opposed to 2–3 weeks in a real XP project. This allows us to have a fair number of iterations, given that the total time that can be spent by the students is very limited. In adjusting to the teaching environment we have tried to look for factors that improve learning. In selecting team size, we have chosen fairly large teams of 8–10 developers. We find this an advantage because it provokes more situations to learn from in a shorter amount of time. For example, the developers will to a larger extent work simultaneously on the same code, provoking merging conflicts to appear more often.

Students are assigned to teams in a random fashion. We think the students learn more this way than when they pick their own team mates which probably have a similar background. This is confirmed by student feedback from the first instance of the course when students asking to be put on the same team were granted that. Several students that were in randomly assembled teams spontaneously commented that they appreciated this. Several students who had selected their teammates reported that they in retrospect would rather have worked in a randomly assembled team.

4. Experiences

Our goal has been to apply the 12 XP core practices more or less according to the book. In addition, we have applied a number of practices, most of which are also described by XP practitioners, but usually not mentioned among the core practices: First Iteration, On-Site Coach, Team In One Room, Spike Time, Reflection, and Documentation. In this section we describe our experiences from applying both the core practices and these added practices. Because we run a number of

iterations we have the opportunity to tune and refine the practices somewhat during the project.

4.1. Experience from XP core practices

4.1.1. Planning game

From the beginning many students were somewhat sceptical at estimating tasks without any prior experience. However, they took on the challenge and gained practical experience on how to estimate new tasks. Most of the groups continued to do estimates and planning right to the end, but some groups did, however, not bother to do estimates towards the end—“we’ll just do tasks and see how far we get”.

Some of the teams estimated in hours and others in relative size of stories. Of the 40 h available (5 pairs for 8 h) during a long lab session, one group consistently got 25–30 estimated hours of work done. Especially the first iteration served to show the students that things took longer than they had thought.

There are a total of 35 stories for all of the 6 iterations, and most teams complete around 30 stories. New stories are introduced for each iteration and on some occasions the customer changes his/her mind on priorities between stories, or changes and extends data formats. This was not planned beforehand the first time we ran the course, but we found it a nice realistic illustration of how customer requirements do change and have kept it for the second iteration of the course.

4.1.2. On-site customer

Since the students are fairly inexperienced programmers and in a learning situation, we want to have good control over the user stories. There should be a sufficient number of stories so that several pairs can work in parallel, and be reasonably simple to implement so that a pair of students have a chance of getting through a story in one iteration (one day of programming). To accomplish this we need customers with XP experience. We therefore settled on not bringing in any outside customer, instead we used faculty as customers. However, we wanted a realistic product where the customer had expert domain knowledge. We settled on a system for tracking races in the motorcycle sport Enduro. One of us (BM) has this sport as his hobby and could provide expertise domain knowledge of how such a system should work for different kinds of races.

In the first instance of the courses, BM served as a single customer for all 12 teams. This proved to be difficult, both because it meant very little time for each team, and because it was difficult to keep track of 12 different products. In the second instance we made use of four faculty as customers, each serving three teams. BM would still serve as the domain authority for the products, available for the customers to discuss with.

We found it very useful to pick a “real” product like this where we had access to real domain expertise, because reality could always be used to guide priorities. For example, the first release was targeted at a version of the product that would supposedly support a race that actually happened in practice (although another system was used at the race). We also had real acceptance test data available from real races run earlier.

We could not afford having the customers on-site at all times. However, they were available by email and phone, and would usually visit their teams twice during each long lab and once or twice during each planning meeting. In our experience, this was sufficient for maintaining a good communication between the customers and the teams. For spontaneous questions and discussions, the coaches would also often act as a stand-in for the customer.

4.1.3. Pair programming

In the first course instance, three of the student coaches performed a thorough study on the Team Programming students’ attitudes and experience from pair programming during the project (Nyström et al., 2002). They conclude with some advice on how to use pair programming in an educational situation:

- The coach can recommend developers to pair in a certain way in order to spread competence and work efficiently. However, it is the developers themselves that ultimately should decide on who to pair with.
- Personality matching seems to play a more important role than competence matching for a pair to work well.
- The coach should strive for assuring that everyone on the team pairs with as many of the others in the team as possible. This is important from a learning perspective and for team building. However, certain pairs might not work well socially, and in such cases they can pair less often and take on simpler tasks in order to not cause bottlenecks for the project.
- Sometimes the two developers in a pair might come into a conflict on how the code should be developed. In this case, the coach can step in and resolve the conflict, or call on a stand up meeting for letting the team as a whole resolve the conflict.

Some teams established a pattern of almost fixed pairs, while others changed partners more often. A few teams even made it a virtue to mandate a change of partner after each task or after the lunch break. There were major differences in programming experience, but in most cases that did not seem to cause problems. The more experienced students were in general happy to share their expertise with the less experienced ones. Sometimes the coach would have to split a pair that

were in crisis with respect to solving a task to get in “new blood”—or to simply reassign the task to another pair.

4.1.4. Collective code ownership and continuous integration

Both these practices were a great success. A common repository was set up using CVS, and each pair would work in their own workspace, doing frequent updates and commits towards the common repository. It quickly became clear to the teams that frequent updates and commits decreased the number and size of merge conflicts.

4.1.5. Small releases

The students do three releases—one every two iterations. The first release includes installation and user manual and is mailed to the customer. In most teams, putting together a release is a lot more work than the students anticipate. There are many things that can go wrong, and it is important to establish a repeatable process. On purpose, we do not give the students detailed instructions for putting together the release, but instead try to get them to think by themselves.

The first release is installed and tested by the customer in the presence of a couple of team members, like in a small informal usability study. The customer “thinks aloud” and the team members observe and note problems of all kinds, e.g., in installing, in reading and understanding the manual, and in using the system. These notes are then turned into small tasks that are entered into the next iteration plan. In many teams the first release went wrong, so they did a new “first” release after iteration three as well (plan to throw one away).

The two subsequent releases also include the source code and technical documentation. These releases are mailed to a peer team for evaluation (and also to the customer). An advantage of this is that the students are forced to see a release from the customer’s point of view.

4.1.6. Metaphor

Metaphor is often said to be XP’s answer to software architecture, and the most difficult of the practices to apply. In the first instance of the course, we did not put much emphasis on the architecture, and as a result many team members had an unclear picture of the design. In the second instance we have tried to remedy this by asking the coaches to make sure that the evolving architecture is being discussed in the team at the planning meetings. Typically, the teams use the naive metaphor from the domain (Race, Drivers, Registrations, etc.) to discuss their architecture, and we did not particularly stress the use of metaphors. The focus on architectural discussions has had a good effect on making the team aware of the big picture, and we will continue to experiment with techniques for improving

this awareness in future instances of the course. One promising technique is the use of metaphors as a discussing technique, as suggested in (Dubinsky and Hazzan, 2003). Here, the goal is not to identify a particular common metaphor, but to use many metaphors in discussing a design. The idea is that the design can be understood better by comparing different metaphors to the system, analyzing which parts they apply to and where they break down.

4.1.7. Simple design and refactoring

The Simple Design practice is difficult to teach because it is very related to experience to understand when a design is simple or not. Furthermore, many students mistake Simple Design with doing the smallest possible changes to the system, i.e., not refactoring sufficiently after making a new feature work. In the second instance of the course we have put more emphasis on Refactoring and we introduced the students to the Eclipse tool, which has support for many refactorings. This has improved the situation substantially, but there is still room for further improvement.

We made it voluntary to use Eclipse or not, but most developers chose to use it (despite that it is rather slow on the fairly old Unix machines we have in the labs), and it had a clear positive impact on the number of refactorings made by the developers.

4.1.8. Coding standard

Most teams adopted the Java coding conventions as the basic formatting standard, and some teams made use of the automatic formatting features of Eclipse. In supporting consistent good names, no common guidelines were applied, and this is an area that might be improved.

4.1.9. Testing

All teams used JUnit and made use of automated tests. Running the automated tests before committing was a standard practice applied. In writing the tests, XP promotes Test First or Test-Driven Development (Beck, 2002) where the typical cycle is (1) write a small test that fails, (2), code to make the test succeed, (3) refactor until the design is simple. Some teams did Test First by the book and some teams almost did it. However, in some cases unit tests were written after the code had been written. In general the students have big problems in understanding how to write unit tests in a useful way. Often some small changes to the code would also require that some unit tests needed to be changed or rewritten because they were too specifically tied to the implementation. At other times there would be unit tests that worked at too high a level and therefore did not catch more subtle bugs.

In the second instance of the course we have focused even more on teaching the small increment Test First

methodology, and asked the coaches to pay particular attention to this. The number and quality of tests have increased, but there is still a lot of room for improvement.

We recommended the students to automate the acceptance tests, but did not demand it from them. Many of the teams learnt a good lesson when things stopped working in the second release that had worked in the first release. This made them realize the need for automated acceptance tests, and most teams implemented such automation by the end of the project.

4.1.10. Forty-hour week/no overtime

In our projects, the 40-h week corresponds to the 8 h day of the long labs on Mondays. In the teaching environment, this use of a fixed time budget rather than fixed requirements was essential to our way of running the course and had a number of important advantages. See “Time Budget” in Section 4.3 below.

4.2. Experience from added practices

In this section we describe how we have applied the added practices and our experience from using them in a teaching perspective.

4.2.1. First iteration

The First Iteration practice (Wake, 2001) says that in the beginning of a project it is very important to build up an executable skeleton of the system, forming an initial thin but complete system. The idea is to make sure early on that the necessary infrastructure can be built and that subsequent development can proceed in small chunks. This idea is somewhat similar to the Elaboration phase in the Unified Process, at least according to the recent interpretations of it as of Craig Larman’s “Agile UP” (Larman, 2001). In XP, First Iteration serves the role of getting early feedback on the system architecture. If some architectural aspect of the system is not working, it is better to evolve the architecture early on before too much time has been invested in building functionality. For this reason, First Iteration is also called “Zero Feature Iteration”.

Building a First Iteration requires experienced programmers and would have been very difficult for our novice students to do. We therefore apply First Iteration by letting the coaches build a tiny first iteration of the system. The Team Programming students can then immediately start adding functionality to the existing system. To aid the coaches in this initial phase we suggest three possible architectures of increasing complexity and suggest that they start with the simplest one and later evolve it to the more advanced ones, if needed.

4.2.2. On-Site coach

The importance of having a coach, in particular when starting out with XP the first time, has been stressed by many XP practitioners (Beck, 2000, Auer and Miller, 2002, Wake, 2001). In a situation of learning or adopting a new set of practices it is important to do a serious job of actually applying the practices. It is all too easy to forget or simply dismiss a practice while concentrating on the actual development. The coach knows the practices, can explain them and motivate why they are important, and can also help the team to actually enforce the practices. Once the practices are mastered the individual is in the position of challenging them and adapting them. We think this is an important way of adopting a new methodology regardless of whether it happens to be XP or something else.

In the accelerated process that we are working with in the project it is important that the coach is always available and actively follows the activities of the team members. Our use of student coaches allows us to afford this. During the long lab sessions, the coaches constantly keep track of what the team members are doing, “stress” them when that is needed, and “caress” them when that is appropriate. An important goal for the coach is team building, to make sure that the team members communicate and collaborate for the common product.

4.2.3. Team in one room

Real XP projects are often set up so that all of the members share an open workspace where program development takes place (Beck, 1999). We adopted this practice: program development takes place only at the long labs where all members of the team are present in the same lab room. We find this to be an extremely important practice.

First of all, the students can easily communicate so that everyone is aware of what is happening, and help from fellow students is always near at hand. For example, when a pair gets stuck they can immediately get help from others on the team, switch tasks, switch pairs, etc., in order to more easily solve the problem at hand.

Secondly, being all in one room allows the coach to easily spot any problems, be it a specific programming task, some pair not following the practices, or a situation when the coach can see that two pairs need to communicate. On such occasions the coach would take a “time-out”, i.e., gather the team for a short stand-up meeting to discuss the problem and find a solution. Rather than telling the developers exactly what to do, the coach would on such occasions ask the developers to come up with a solution. This way, the coach helps the developers to learn to help each other, and soon the developers would themselves call on stand-up meetings when needed.

Thirdly, being all in one room very strongly promotes team building. The teams are put together randomly so initially the students do not know each other well. However, due to all the interaction during the long labs, most of the teams soon develop a very strong social team spirit, which again promote further communication and awareness.

It may not always be easy to schedule full day labs for students who take several classes at the same time. However, since these are 2nd year students, most of their schedule is still fixed by mandatory courses, and it has been possible for us to negotiate a schedule that do not interfere with other courses. Scheduling is a bit of a problem for some of the student coaches, but this is handled by letting them coach in pairs, so that at least one coach is always present for each team.

4.2.4. Spike time

In addition to program development, the developers need time to do individual work, e.g., to read and learn about various practical issues, to think about how to solve a particular task, or to do experimental programming with new solutions. We term this “spike time”. During each planning meeting, the team allocate this spike time in the way they think best for the team as a whole, thereby providing a pool of additional resources. We give the coaches some initial ideas for how to allocate the spike time, and after some iterations the following patterns emerge:

- *Spiking for a task.* In order to be able to program a particular task at the long lab, the students experiment with some key aspects, for example find out what Java libraries to use, and write or find some simple example code that can be used as a starting point later in the long lab.
- *Reading/learning about tools.* In some teams, a “CVS expert” is appointed whose job it was to use all his/her spike time for learning about CVS in order to help the teammates at the long lab. In other teams, all team members were allocated a couple of hours for experimenting with CVS. Similar spiking was done for other basic tools like JUnit and makefiles, and also for tools introduced during the project such as an acceptance-testing tool for one team.
- *Refactoring.* After a couple of iterations many teams find the need for doing large refactorings of the code. This is difficult to do during the long labs since it affects and/or delays many teams and produces many merge conflicts. Several teams adopted the policy of allocating spike time for such refactorings. In these cases, a single pair usually did the refactorings, or a few pairs would take turns in doing refactorings.
- *Code review and unit tests.* Some teams found that they had too few tests, and used spike time for doing

systematic review in order to find code with missing unit tests and to remedy that.

For each team an additional CVS repository is set up that includes results from the spikes. This way, the developers can easily share their results with their team members. Not only code is checked in, but also short manuals and other material that the developers produced during their spike time. The general rule was to not check in to the regular repository during spike time, since changes there should only happen when all the team members are present at the long labs. However, many teams developed the practice that certain tasks that were allocated to spike time, e.g., big refactorings and adding test code, could be checked after agreeing beforehand at the planning meeting.

The coach meetings (when all coaches met and discussed the previous and next iteration) allow the coaches to share ideas for how to use the spike time and result in many new ideas that we had not been able to foresee. At the last iterations, there was less need for spike time because development had become very intense, and the developers needed more time for development rather than for spikes for new tools or new tasks. However, there was always the need for increased code quality, which motivated e.g. spike time for writing more test cases and refactoring.

4.2.5. Reflection

The Agile Manifesto (Manifesto, 2001) lists reflection as one of the principles in order to follow an agile methodology: “At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly”. In our XP project, there were several occasions when such reflection occurred spontaneously, e.g. at stand up meetings called on by the coaches in order to discuss a particular practice, and at the planning meetings when the results of the previous iteration were discussed. On such occasions it was often decided to do a spike to avoid that a problem repeated itself. It is our experience that the spontaneous reflection greatly helped in team building, as well as improving the effectiveness of the team.

In the second instance of the course we advised the coaches to initiate reflective discussion over the team work practices at the planning meetings. A more structured way of doing that would be to use project retrospectives as suggested in (Kerth, 2001), something we might try in the next instance of the course.

4.2.6. Documentation

XP does not prescribe any specific documentation, but views the code as the most important (and always up-to-date) documentation. However, core XP can of course be extended with additional practices to cater for appropriate additional documentation. In our XP pro-

ject, we let the customer write a user story demanding technical documentation for the system, including an architectural description, build description, description over internal file formats, etc. This is very easy to motivate since the customer might like to take the finished source code and let another team develop it further at a future time.

Several teams also spontaneously developed additional internal documentation like instructions for how to do a release, story test descriptions, etc. Part of this documentation was developed during spike time.

4.3. Lessons learned

We have learned many things from running the Team Programming course in this new format, and we summarise our most important lessons below.

4.3.1. Team-In-One-Room is fundamental

Of all the additional practices we introduce, the practice of having the whole team in one room is perhaps the most important of all. We have heard of other experiments in applying XP in education where the teams are just left to themselves to find a common time and place for programming. This usually results in the team splitting into fixed pairs, splitting the stories between them and programming them at different times without much communication, learning, or reflection. In these cases they usually also get problems with collective code ownership and resort to code owning practices (Bendix and Hedin, 2002). We view Team-In-One-Room as a fundamental practice to apply in an educational setting, and that if it is not possible to schedule full day lab sessions like we did, splitting into a couple of half day lab sessions is better than splitting the team.

4.3.2. Coaching by older students

In novice teams there is a need for someone with more experience who can take an overall perspective on both the product architecture and the team practices. We had very good experience from using older students as coaches. This way the coaches have authority both due to their general seniority and in that they are given special training in the Coaching course. Yet, they are not too distant socially from the Team Programming students, which make it easy for them to build a good team spirit. The coaches also found it very stimulating to take on the needed responsibilities.

4.3.3. Time budget

As mentioned earlier, the students are required to spend a certain number of hours on the project, rather than completing a certain number of user stories. This has been more important than we initially anticipated,

bringing a number of advantages in the teaching situation.

- Experienced students were willing to pair with inexperienced students and were happy to show off their expertise, allowing the others to learn.
- The students could focus on code quality, writing tests and refactoring for simplicity, rather than on just completing functionality. However, the general tendency is still to focus on completing functionality, and focusing on code quality has to be quite intensively coached. As a small success story, one of the coaches reported that he on one occasion had a highly competent pair that for the moment were out of high-priority tasks. He suggested that they would spend some time looking over the code and improve it by refactoring it. At first, they were very reluctant since it seemed dull to them. However, once they got started on this refactoring task, they were soon drawn into it, engaging themselves seriously, and obviously enjoying themselves a lot.
- The time-budget set up allowed us to change and add tasks continuously through the project. This included not only user stories, but also to schedule various things that were of benefit for the team, e.g., writing checklists, trying out new tools, etc. The students were never upset about tasks being changed or added since it did not affect the total amount of time they spent on the project.

4.3.4. Underestimated difficulties for the students

During the course it became apparent to us that we had underestimated the difficulty of several practices.

- The “Test First” practice was much more difficult to explain and teach than we initially thought. There is a need for very clear examples on how to write tests, we have improved this in the second instance of the course, but it can be improved even more.
- The “Simple Design” practice was also more difficult to teach than we expected. Some students misinterpreted “Do the simplest thing that can possibly work” to mean “Change as little as possible to incorporate a new feature”, resulting in a bad design. Here, there is again a need for many good examples (and maybe also anti-examples) and to follow up this better during the project. The situation has improved in the second instance of the course, partly due to more stress on this in the theory part, and partly due to the introduction of refactoring tools. However, this is still a difficult practice that students need a lot of experience to master.
- Both the students and we underestimated their problems in putting together a release. Fortunately though, we had scheduled for three releases. Several students thought it would take around 15 min to pro-

duce a release, whereas in reality it took up to half a day for a pair, and many of the first releases did not even work. After this experience many of the teams allocated a pair at the next iteration to put together a “release process”, i.e. to build a checklist for how to put together a release and what things to check in order to be assured that it would work. Once that was in place, the release process went very smoothly for most teams and they could often produce a new release in a matter of minutes.

Although we underestimated many difficulties and the students ran into many problems during their project, this is not necessarily a bad thing. On the contrary, we find it very valuable for the students to run into these problems so they can better appreciate the solutions. Due to the many iterations they get experience from both having problems and solving them, which provides a good basis for learning. The preceding theory part of the course together with the coaching during the project do, however, provide the students with a reasonable background and framework so they do not run into all problems at once, but can feel that they make progress with each iteration.

4.3.5. Being a customer is much work

We underestimated how much work it is to be a customer. We need a good amount of stories that are not too difficult and that can be done in parallel by several pairs. Due to the very short 1-day iterations the stories need to be much smaller than for a real XP project. To accomplish this we broke down the larger stories into smaller tasks, although this is something that normally the team would do itself, and we also gave the teams directions for the initial architecture and possible ways of evolving the architecture. We used the same product during the second instance, but gave slightly different priorities to the stories. As we get more experience with the course we might get bolder and bring in new products on the fly, and put more responsibilities on the team like breaking down stories to tasks and giving them more responsibilities for coming up with the architecture.

4.3.6. XP might have positive effects for minority students

The high interaction and collaboration within an XP team, in particular when guided by a trained coach, might have positive effects for students in minority groups. At the computing engineering program, only around 10% of the students are women. Most women on the course had very positive experience from the course, and anecdotal evidence points towards them more easily being able to take on active roles as compared to earlier courses. This is confirmed by other studies (Bullock et al., 2003).

5. Coaching practices

Based on our experiences from the courses, we have developed a number of coaching practices that are discussed briefly below.

5.1. Coach

The coach should know the basic XP practices and be able to explain and motivate them to the team. Being a slightly more senior student, the coach also sometimes has technical knowledge that is of help to the team. However, the coach does not know everything, so the main role of the coach is to help the team to: learn the XP practices, develop the practices, learn to communicate, and to help them learn from each other. In essence, the goal of the coach is to help the team to help themselves. This is done by being an *ActiveTrackingCoach* and by *TriggeringCommunication*.

5.2. ActiveTrackingCoach

The coach is active in the sense that he/she actively follows the activities of the team, and in practice works as a tracker. The coach keeps track of what each programming pair does, how their work progresses, and how the code evolves. This allows the coach to act proactively, already when problems start to arise, rather than after the fact. If the coach spots a problem, it can often be solved by *TriggeringCommunication* between team members.

5.3. TriggerCommunication

In a team learning XP, communication within the team often does not arise spontaneously, but needs to be learnt. Initially, the team members do not know each other well and are hesitant to communicate. Also when they know each other well, they may be hesitant to ask for help because they are not yet a functioning team. A typical sign is that a pair does not ask for help when they have gotten stuck on some technical problem. Sometimes, the coach can help to solve the problem, but more often, the coach can explain that it is a practice in this case to ask other programmers on the team for help, or to switch pairs. Another common case when communication is needed is when two pairs are working on the same code without being aware of this. If a problem is of relevance to all of the team, the coach can take a *TimeOut*.

5.4. TimeOut

For any kind of problem that occurs in the team, if the coach cannot solve it easily and directly, a good way

of tackling the problem is to take a *TimeOut*, i.e., to bring the team together for a brief stand-up meeting, and let the team solve the problem together. This works for technical problems, where someone on the team might have skills in a certain area, or where the team can brainstorm possible solutions or ideas for experimental spikes for solving the problem. A *TimeOut* is also useful for process problems where the team needs to agree on what practices to follow. Basic XP practices are given by the course material, but the team needs to reason about these practices in order to understand and accept them. There are also lots of little sub practices that need to be developed by the team itself. Often such sub practices emerge spontaneously and can be turned into a more permanent practice at a *TimeOut*. As an example, several teams developed the practice of switching pair programming partners after lunch. This practice had the advantage of getting the team members to know each other in a short time, and turned out to be logistically easier in the scaled down project than the normal XP practice to switch partner after the completion of a task or story.

5.5. PairCoaching

Our students coach in pairs. While this might be inefficient in a commercial setting, it has many benefits in an educational environment. The coaches appreciate having someone to share the coaching with. It also allows a greater number of students to take the Coaching course, a greater flexibility in the exact number of coaches needed, and finally it gives a redundancy that is good to have in case of temporary illness, since the coaches are crucial to the team.

5.6. CoachIsCoached

On every iteration, between the long lab and the next planning meeting, all coaches meet for 2 h of reflection and advice. During this meeting, the coaches share their experience from the previous long lab, and many good ideas emerge among the participants on how to handle various kinds of problems. These meetings are coached by faculty who also give advice on what to focus on during the next iteration. For example, the first iteration might focus on just getting started; the second one on team building, e.g., by switching pairs often; the third one on making sure that a swift release process is developed; the fourth one on checking extra carefully that testing is done in an appropriate way; etc. What to focus on need not be fixed in advance, but can be a reaction to what kind of problems that arose in the previous iteration.

5.7. *CoachIsArchitect*

The students taking the XP course have previously studied programming, algorithms and data structures, and object-oriented modelling and design, but have previously only had assignments of small programs of a few hundred lines of code. They are thus quite inexperienced programmers. The product to be developed by the team is, in general, their first larger program, and there is a need for someone on the team to take a more senior role concerning the software architecture. We solve this by letting the coach take on the role as an architect as well.

During the project, the coaches have the responsibility of taking the initiative to architectural discussions during the planning meetings; they keep track of the evolving architecture, and document it at the end of the project. This allows the coach to practice software architecture in a realistic, albeit small, project.

The architect role here is very different from the traditional one of prescribing an architecture before implementation. Here, the architect role is rather to take the responsibility for keeping track of the architecture, verbalizing it and maybe visualizing it to allow it to be communicated among team members and, if needed, to take the initiative to architectural refactorings.

6. Related work

There are several experiments on including some or all of the XP practices into the undergraduate education, e.g. (Astrachan et al., 2003, Johnson and Caristi, 2003, Holcombe et al., 2003, Schneider and Johnston, 2003, Mugridge et al., 2003). Müller and Tichy (2001) performed a study of two small 4th year student XP teams, focusing primarily on evaluating the XP practices. They confirm our observation that coaching is very important. Noll (2002) provides some observations from initial experience on applying XP to student projects, confirming our experience that XP is an excellent aid in learning, due to its highly iterative nature, allowing students to make mistakes and learn from them. Both these studies also confirm our experience that the Test First practice is difficult to learn. Our study differs by being much larger in scope, both in the number of students and teams, and in the number of additional practices applied. Our study also differs in being focused on XP as an aid in learning software engineering concepts rather than finding out if XP as such is efficient or not.

Williams and Upchurch (2001) provide some guidance on the use of the XP practices in an educational context. Most of their advice is in agreement with our experience. However, their advice on not using collective code ownership for novice students is in contrast to our

findings. We find collective code ownership to be very important and not problematic at all. All our teams use CVS and are taught to synchronise frequently. In combination with the practice of keeping the whole team in one room, this is probably the reason for our success with collective code ownership. Some teams did initially not synchronise frequently, but the merge problems they then got made them understand how important this is.

7. Conclusions

When our course in introductory Software Engineering needed to be revised we set out to try to teach basic principles applicable on the team level and to use XP as the method. Other secondary goals of the course is to get the students more experienced as programmers, to better understand the problems of working in a team with others, interacting with a customer/user and to expose them to some new programming tools. The Team Programming has been taught twice in full scale with more than 100 students each time with a background in fundamentals of programming, but no prior training in Software Engineering. The course is followed by other courses giving a more traditional software engineering perspective.

In contrast to earlier versions of the course we decided to take a larger responsibility for the management of the project part of the course, actually teaching a method, enforcing and supporting it during the project. For us it was a very interesting alternative to use XP as the method that we judged would suit this situation and would be a vehicle to expose the students to some fundamental software engineering principles and practices.

The results are very positive: we feel that the students have a good understanding for fundamental problems and techniques in software development in mid size teams including testing, inspection, interaction with user/customer, requirements, and release process. They also understand how to work in an iterative method and are exposed to the problems of changing requirements. During the project they experience the problems of being many developers changing the same source and some mechanisms to handle these: configuration management, merge, short checkout times, refactoring and separated tasks. The students give very positive feedback after the course—it was at times very intense for them, but they report that they learned a lot and are very satisfied with the course.

Our experience is also that XP works very well for this setting, introducing practices and techniques that can be motivated directly by the situation the students are facing. Having a blank table to start with, we set out to use as much of XP practices as we found motivated, expecting that they would support each other. We did, however, have to do some amendments

and additions due to the particular teaching situation. The most important addition we feel is to provide support for the architecture, having to do with rather inexperienced students, and thus providing a first iteration from the start and putting more of the responsibility for the software architecture on the coaches. We also broke up the work in smaller tasks than customary in order to have the students perform many iterations.

The format of the course needed to be adjusted to the use of the XP method and in particular we managed to schedule one full day each week for software development. We thus have the students in the same room for 8 h and can use pair programming with changing partners. Only during this time is it allowed to develop production code. The set up also is very important in order to build a team and to get all the participants to actually know each other well enough. Another unusual set up is to use students from another SE course as coaches for the teams. This allows for much more intensive coaching than would have been possible using ordinary faculty resources, and provides a realistic setting for the coaches to learn practical aspects of architecture, team building, and leadership.

In this educational setting, there is a need for a streamlined process for learning how to coach, and development of coaching practices. We have described the main practices that we developed to this end: Coach, ActiveTrackingCoach, TriggerCommunication, Time-Out, CoachIsCoached, PairCoaching, and CoachIsArchitect.

The coaching role still has the association of self-learned guru in the XP world. Our experience shows that it is quite possible to teach coaching in a systematic way, even to fairly inexperienced programmers. The coaching practices we have developed to this end will certainly evolve as we get more experience, and should be thought of as proto practices that need to be further elaborated and evaluated. Nevertheless, we have tried them out with success and believe that other educators will be able to use the same techniques. The coached teams were very satisfied with their student coaches, and the coaches themselves found the course very interesting and stimulating.

Although this paper is about teaching XP to undergraduate students, we think many of the lessons learned apply also to introducing XP in industry. So far, many companies have adopted XP for voluntary teams in pilot projects. To introduce the methodology to a larger number of people, it might well be advantageous to do it more like a course, doing a scaled down project with many iterations, coach-on-site, team-in-one-room, and trying to do it “by the book”. Because the developers are learning new tools and practices, they will initially go slower than they are used to, and viewing this as a course with a fixed time budget might make it easier to

try out the new techniques seriously without falling back into old habits.

Acknowledgements

The development of these courses has been supported by many people at the department. In particular we would like to thank Torbjörn Ekman and Christian Andersson who came up with the initial idea of full day lab sessions, and for being very helpful in the implementation of the courses. Thanks also to all the coaching students, in particular for all ideas and feedback that surfaced at the coaching meetings.

References

- Astrachan, O.L., Duvall, R.C., Wallingford, E., 2003. Bringing Extreme Programming to the Classroom, in *Extreme Programming Perspectives*. Addison-Wesley.
- Auer, K., Miller, R., 2002. *Extreme Programming Applied*. Addison-Wesley.
- Babich, W.A., 1986. *Software Configuration Management—Coordination for Team Productivity*. Addison-Wesley (Chapter 1).
- Beck, K., 1999. *Embracing Change with Extreme Programming*. IEEE Computer (October).
- Beck, K., 2000. *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Beck, K., 2002. *Test-Driven Development*. Addison-Wesley.
- Beck, K., Gamma, E., 2002. *JUnit Cookbook*. Available from <junit.sourceforge.net/doc/cookbook/cook-book.htm>.
- Bendix, L., Hedin, G., 2002. Summary of the Subworkshop on Extreme Programming. *Nordic Journal of Computing* 9 (3).
- Bullock, H., Fernald, J., McDowell, C., Werner, L., 2003. The impact of pair programming on student performance and pursuit of computer science related majors. In: *Proceedings of the International Conference on Software Engineering*, Portland Oregon.
- Demarco, T., Lister, T., 1999. *Peopleware: Productive Projects and Teams*, second ed. Dorset House.
- Dubinsky, Y., Hazzan, O., 2003. Using metaphors in extreme programming projects. In: *Proceedings of XP 2003 LNCS Series*. Springer-Verlag.
- Fowler, M., 1999. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., Vlissides, J., 1993. Design patterns: abstraction and reuse of object-oriented design. In: *Proceedings of ECOOP LNCS 707*. Springer Verlag, pp. 406–431.
- Hedin, G., Bendix, L., Magnusson, B., 2003. Introducing software engineering by means of extreme programming. In: *Proceedings of the International Conference on Software Engineering*, Portland Oregon, 5–8 May 2003.
- Hedin, G., Bendix, L., Magnusson, B., 2003b. Coaching coaches. In: *Proceedings of XP 2003 LNCS Series*. Springer-Verlag.
- Holcombe, M., Gheorghe, M., Macias, F., 2003. Teaching XP for Real: Some Initial Observations and Plans, in *Extreme Programming Perspectives*. Addison-Wesley.
- Jeffries, R., Anderson, A., Hendrickson, C., 2001. *Extreme Programming Installed*. Addison-Wesley.
- Johnson, D.H., Caristi, J., 2003. Extreme programming and the software design course. In: *Extreme Programming Perspectives*. Addison-Wesley.

- Kerth, N.L., 2001. *Project Retrospectives: A Handbook for Team Reviews*. Dorset House.
- Kruchten, P., 1995. The 4+1 view model of architecture. *IEEE Software* 12 (6), 42–50.
- Larman, C., 2001. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*, second ed. Prentice-Hall.
- Manifesto for Agile Software Development, 2001. Available from <agilemanifesto.org/>.
- Mugridge, R. et al., 2003. Five challenges in teaching XP. In: *Proceedings of XP 2003 LNCS Series*. Springer-Verlag.
- Müller, M.M., Tichy, W.F., 2001. Case study: extreme programming in a University environment. In: *Proceedings of ICSE 2001*, Toronto, Canada.
- Noll, J., 2002. Some Observations of Extreme Programming for Student Projects, position paper at the Workshop on Empirical Evaluation of Agile Processes, Chicago, IL, 7 August 2002.
- Nyström, M., Rix, J., Wanhainen, K., 2002. En studie om parprogrammering i praktiken (in Swedish). Available from <www.cs.lth.se/Education/LTH/01.Dokt.XP/Djupstudier/NystromRixWanhainen.pdf>.
- Schneider, J., Johnston, L., 2003. Extreme programming at Universities—an educational perspective. In: *Proceedings of ICSE*. The Eclipse Java Development Tools subproject. Available from <www.eclipse.org>.
- Wake, W.C., 2000. The Test Code Cycle in XP: Part 1, Model, <www.xp123.com/xplor/xp0002>.
- Wake, W.C., 2001. *Extreme Programming Explored*. Addison-Wesley.
- Whitmore, J., 2002. *Coaching for Performance: Growing People, Performance and Purpose*, third ed. Nicholas Brealey.
- Williams, L., Upchurch, R., 2001. Extreme programming for software engineering education? In: *Proceedings of the 31st ASEE/IEEE Frontiers in Education Conference*, Reno, Nevada, 10–13 October 2001.
- Görel Hedin** is an associate professor of Computer Science at Lund University, Sweden. Her research interests include object-oriented languages and their implementation, object-oriented design, and agile methodologies. She has served on the program committees of several international workshops and conferences including ECOOP and CC. In 2003 she was the program chair for CC. She received a Ph.D. in Computer Science from Lund University in 1992.
- Lars Bendix** is a visiting professor at Lund University, Sweden. His main research interest is software configuration management and how it can be used to support software development processes. He is also interested in agile processes and their pedagogical use in software engineering education. He received a Masters Degree in Computer Science from Aarhus University, Denmark in 1986 and a Ph.D. Degree from Aalborg University, Denmark in 1996.
- Boris Magnusson** is a professor of Computer Science at Lund university, Sweden, and is the director of the Center for Applied Software Research at Lund University. His research interests include object-oriented systems and implementation, agile methodologies, configuration management, and embedded systems. He has served on the program committees of numerous international conferences including OOPSLA, ECOOP, TOOLS, SCM, ESEC, and ICSE. In 2002 he was the program chair for ECOOP. He received a Ph.D. in Computer Science from Lund university in 1984.